

Ecole Marocaine des Sciences de l'Ingénieur

Rapport de Stage de Fin d'Année

**Clarté — Conception et développement d'un chatbot intelligent
basé sur les techniques Retrieval-Augmented Generation (RAG)
pour l'exploitation de documents d'entreprise**

Réalisé par : *HASSANI SAFAA*

Tuteur(s) :

Encadrant Professionnel : *ZAOUATI SAFAA*

Encadrant Pédagogique : *MOUSSAB BATAL*

Au sein de : OCP Group (Digital & AI Office)

Filière : Intelligence Artificielle & Data Science — 4ème Année

Année universitaire : 2025/2026

Dédicaces

À mes chers parents, pour leur amour inconditionnel, leurs sacrifices, leurs encouragements et leur soutien indéfectible tout au long de mon parcours. Aucune dédicace ne saurait exprimer ma profonde gratitude envers vous. Que ce travail soit le fruit de vos efforts et le témoignage de ma reconnaissance éternelle.

À mes frères et sœurs, pour leur affection, leur présence et leurs encouragements constants qui m'ont toujours donné la force d'avancer.

À toute ma famille, pour leur soutien moral et leurs prières qui m'ont accompagné à chaque étape de ce chemin.

À mes amis, pour les moments partagés, les encouragements et le soutien mutuel durant ces années d'études. Votre amitié a été un véritable pilier tout au long de ce parcours.

À toutes les personnes qui, de près ou de loin, ont contribué à la réussite de ce travail. Je vous dédie ce travail, avec toute ma gratitude et mon affection.

Remerciements

Je tiens, en premier lieu, à exprimer toute ma gratitude à Madame Safaa ZAOUATI, mon encadrante professionnelle, pour la disponibilité dont elle a fait preuve, la pertinence de ses conseils ainsi que le suivi rigoureux qu'elle a assuré tout au long de ce stage au sein de l'OCP Group.

Je souhaite également adresser mes remerciements les plus sincères à Monsieur Moussab BATAL, mon encadrant pédagogique, pour la qualité de son accompagnement, la clarté de ses orientations méthodologiques ainsi que la pertinence de ses observations, qui ont grandement contribué à l'amélioration de ce travail.

Mes remerciements s'adressent par ailleurs à l'ensemble des collaborateurs de l'OCP Group ayant, de près ou de loin, participé à la bonne réalisation de ce projet, ainsi qu'au corps professoral pour la qualité de la formation dispensée tout au long de ce cursus.

Enfin, je tiens à exprimer ma profonde reconnaissance envers ma famille et mes proches, pour le soutien constant et bienveillant qu'ils m'ont témoigné durant cette période.

Table des matières

Dédicaces	1
Remerciements	2
Table des figures	5
Liste des tableaux.....	6
Introduction générale.....	7
Chapitre 1.....	8
1.1 <i>Introduction.....</i>	8
1.2 <i>Présentation de l'OCP.....</i>	8
1.3 <i>Étude de l'existant.....</i>	9
1.3.1 Description de l'existant	9
1.3.2 Problématique	9
1.4 <i>Objectifs du projet.....</i>	10
1.4.1 Objectif général.....	10
1.4.2 Objectifs spécifiques	10
1.4.3 Solution proposée.....	11
1.5 <i>Choix du modèle de développement</i>	11
1.5.1 Méthodologie du travail	11
1.5.2 Méthodologie Agile pour Clarté	12
1.6 <i>Planning prévisionnel.....</i>	13
1.7 <i>Conclusion.....</i>	13
Chapitre 2.....	14
2.1 <i>Introduction.....</i>	14
Contexte et problématique	14
2.2 <i>Spécification des besoins fonctionne.....</i>	14
2.2.1 Authentification et gestion des comptes.....	15
2.2.2 Import et gestion des documents	15
2.2.3 Extraction et indexation des documents.....	15
2.2.4 Recherche sémantique.....	15
2.2.5 Génération de réponses (chat RAG).....	16

2.2.6 Historique des conversations	16
2.2.7 Administration.....	16
2.3 <i>Spécification des besoins non fonctionnels</i>	17
2.4 <i>Choix d'architecture — Le pipeline RAG</i>	18
2.5 <i>Présentation des cas d'utilisation</i>	20
2.5.1 Présentation des acteurs	20
2.5.2 Description des cas d'utilisation	20
2.5.3 Diagramme des cas d'utilisation global	22
Chapitre 3.....	24
3.1 <i>Introduction</i>	24
3.2 <i>Modélisation dynamique</i>	24
3.2.1 Diagramme de séquence — Authentification.....	24
3.2.2 Diagramme de séquence — Question/réponse RAG	25
3.2.3 Diagramme d'activité — Import et indexation d'un document	27
3.3 <i>Modélisation statique</i>	28
3.3.1 Diagramme de classes.....	28
3.3.2 Modèle relationnel	29
3.3.3 Dictionnaire de données.....	29
3.3.4 Architecture de l'application.....	30
3.4 <i>Conclusion</i>	32
Chapitre 4.....	33
4.1 <i>Introduction</i>	33
4.2 <i>Environnement de développement</i>	33
4.2.1 Environnement matériel	33
4.2.2 Environnement logiciel	34
4.3 <i>Principales interfaces graphiques</i>	38
4.4 <i>Conclusion</i>	43
Conclusion générale	44
Bibliographie.....	45

Table des figures

Figure 1 - Planning prévisionnel.....	13
Figure 2 - Diagramme de cas d'utilisation globale.....	22
Figure 3 - Diagramme de séquence — Authentification	24
Figure 4 - Diagramme de séquence — Rag	25
Figure 5 - Diagramme d'activité.....	27
Figure 6 - Diagramme de classes	28
Figure 7 - Diagramme de composant de l'architecture logicielle de Clarté.....	31
Figure 8 - Diagramme de déploiement	32
Figure 9: FastAPI.....	34
Figure 10:LangChain	34
Figure 11:ChromaDB.....	35
Figure 12: Sentence Transformers	35
Figure 13:Llama 3.....	35
Figure 14: SQLite	36
Figure 15:React.js	36
Figure 16:Bcrypt.....	37
Figure 17:JWT	37
Figure 18: Git.....	37
Figure 19:Visual Studio	37
Figure 20: Page d'accueil	38
Figure 21: Inscription.....	39
Figure 22:Connexion	39
Figure 23:Interface de chat	40
Figure 24:Page de recherche sémantique.....	40
Figure 25:Rapport génère	41
Figure 26:Tableau de bord administrateur	41
Figure 27:Test de réalisation.....	42

Liste des tableaux

Tableau 1 - Synthèse des besoins fonctionnels	14
Tableau 2 - Synthèse des besoins non fonctionnels	17
Tableau 3 - Synthèse des acteurs	20
Tableau 4 - Description du cas d'utilisation « Poser une question » pour l'acteur Utilisateur	21
Tableau 5 - Description du cas d'utilisation « Gérer les utilisateurs et les documents » pour l'acteur Administrateur	22
Tableau 6 - Dictionnaire de données - table users	29
Tableau 7 - Dictionnaire de données — table documents.....	30
Tableau 8 - Dictionnaire de données - table conversations.....	30
Tableau 9 - Dictionnaire de données - table messages	30
Tableau 10 - Environnement logiciel.....	34

Introduction générale

Réalisé au sein de l'OCP Group, ce projet s'inscrit dans un contexte industriel où les collaborateurs doivent consulter régulièrement un volume important de documents internes : politiques RH, procédures, comptes rendus et notes de service. Ces documents constituent une source d'information essentielle pour les activités quotidiennes et peuvent comporter des données sensibles.

Dans le périmètre observé, l'existant repose principalement sur l'identification manuelle du document, la navigation dans les espaces de stockage disponibles et la recherche par mots-clés. Lorsque la formulation de la question diffère du vocabulaire employé dans le document, ou lorsque plusieurs versions coexistent, cette méthode impose de parcourir de nombreux fichiers. Elle allonge le délai de recherche, dépend de la connaissance des dossiers par le collaborateur et conduit fréquemment à solliciter un collègue ou le service concerné.

Le projet Clarté vise à répondre à ces limites par un chatbot intelligent fondé sur les techniques de Retrieval-Augmented Generation (RAG). Cette approche associe la recherche sémantique, qui retrouve les passages les plus pertinents à partir d'une question formulée en langage naturel, et la génération de texte par un modèle de langage. Les réponses sont ainsi contextualisées et accompagnées de sources vérifiables.

Se pose alors la problématique suivante : comment permettre à un collaborateur d'obtenir, en langage naturel et en quelques secondes, une réponse fiable et sourcée à partir des documents internes de l'entreprise, sans dépendre d'un service externe qui exposerait des données sensibles hors de l'organisation ?

Afin de répondre à cette problématique, le présent rapport est organisé en quatre chapitres. Le premier chapitre présente le cadre général du projet, à savoir l'organisme d'accueil, l'étude de l'existant ainsi que la solution envisagée. Le deuxième chapitre est consacré à la spécification des besoins fonctionnels et non fonctionnels, ainsi qu'à l'identification des cas d'utilisation. Le troisième chapitre expose la conception du système, tant du point de vue dynamique que statique. Enfin, le quatrième chapitre décrit la réalisation concrète de l'application, l'environnement de développement utilisé, ainsi que les principales interfaces graphiques obtenue.

Chapitre 1

Présentation du cadre de projet

1.1 Introduction

Ce chapitre a pour objectif de situer le projet dans son contexte général. Nous y présentons, dans un premier temps, l'organisme d'accueil ainsi qu'une étude de l'existant en matière de gestion et de consultation des documents internes. Cette étude est suivie d'une critique de l'existant, mettant en évidence les limites justifiant la mise en place d'une nouvelle solution. Nous exposons ensuite la solution proposée ainsi que le modèle de développement retenu pour la conduite du projet. Ce chapitre se conclut par la présentation du planning prévisionnel suivi durant la période du stage.

1.2 Présentation de l'OCP

L'OCP Group (Office Chérifien des Phosphates) est une entreprise marocaine fondée en 1920, positionnée comme leader mondial sur le marché des phosphates et de leurs dérivés, notamment les engrais. Le groupe intervient sur l'ensemble de la chaîne de valeur, depuis l'extraction du phosphate brut au niveau de ses sites miniers de Khouribga, Benguerir et Youssoufia, jusqu'à sa transformation industrielle dans les complexes chimiques de Jorf Lasfar et de Safi, avant l'exportation des produits finis vers différents marchés internationaux. Le groupe emploie plusieurs dizaines de milliers de collaborateurs et occupe une place importante dans l'économie marocaine.

Notre stage s'est déroulé intégralement à distance, dans le cadre d'une collaboration avec l'OCP Group. Cette organisation en télétravail n'a pas empêché un suivi régulier du projet : celui-ci a été assuré directement par nos encadrants professionnel et pédagogique, à travers des points d'avancement réguliers tout au long du développement. C'est dans le cadre de ces échanges que nous avons pris connaissance du besoin exprimé de faciliter l'accès aux informations contenues dans les documents internes de l'entreprise — besoin qui a directement motivé la définition du projet Clarté.

1.3 Étude de l'existant

1.3.1 Description de l'existant

Au sein de l'organisme d'accueil, la consultation des documents internes (politiques, procédures, notes de service) repose aujourd'hui sur des méthodes classiques : partages réseau, arborescences de dossiers organisées par service ou par thème, ou moteurs de recherche interne limités à la recherche par mots-clés exacts. Ces documents sont, pour la plupart, rédigés au format PDF ou Word, et regroupés par catégorie (ressources humaines, procédures internes, notes de service, comptes rendus), sans mécanisme centralisé permettant d'interroger leur contenu en langage naturel.

Concrètement, un collaborateur souhaitant connaître, par exemple, le nombre de jours de congé maladie ou les modalités de télétravail doit d'abord identifier le document susceptible de contenir cette information — ce qui suppose de connaître, ou de deviner, dans quel dossier ou sous quel intitulé il a été classé — puis l'ouvrir et le parcourir intégralement, parfois sur plusieurs dizaines de pages, jusqu'à repérer le passage pertinent. Lorsque le document recherché n'est pas immédiatement localisable, il n'est pas rare que le collaborateur se tourne vers un collègue ou vers le service concerné pour obtenir l'information oralement ou par messagerie, ce qui déplace la charge de recherche sur un tiers sans garantir que la réponse obtenue soit à jour ou exacte. Cette situation est d'autant plus sensible lorsque plusieurs versions d'un même document coexistent (mises à jour de procédures, par exemple), sans qu'il soit toujours évident de savoir laquelle fait référence.

1.3.2 Problématique

Les constats présentés précédemment conduisent à formuler la problématique centrale de ce mémoire de la manière suivante : comment concevoir et déployer Clarté, une plateforme web d'assistance documentaire et décisionnelle, capable d'exploiter automatiquement des corpus d'entreprise hétérogènes pour fournir des réponses pertinentes, traçables et actionnables aux équipes commerciales et marketing, tout en respectant les contraintes opérationnelles et de sécurité propres au milieu industriel ? Cette problématique soulève plusieurs questions opérationnelles et techniques qui structurent la démarche de ce mémoire : comment automatiser l'ingestion et le prétraitement de documents (PDF, DOCX, TXT) en garantissant une qualité d'extraction suffisante malgré l'hétérogénéité des formats et des qualités de numérisation ? Comment indexer et rechercher efficacement des passages pertinents tout en évitant les faux positifs introduits par une recherche top-k purement statistique (nécessité d'un seuil de pertinence mesurable et de critères de confiance) ? Comment construire un dialogue en langage naturel dont les réponses sont exclusivement fondées sur les documents importés, assorties de preuves et de sources explicites ? Quels compromis d'architecture, de gouvernance et de déploiement (prototype local, environnement de test, architecture cible en entreprise) sont nécessaires pour réduire les risques de fuite de données sans présenter l'exécution locale comme une garantie absolue de confidentialité ? Enfin, comment évaluer de manière objective la fiabilité, la pertinence et l'utilisabilité des informations produites par la plateforme dans des scénarios métier réalistes ?

1.4 Objectifs du projet

1.4.1 Objectif général

L'objectif général du projet est de concevoir, développer et évaluer Clarté : une plateforme d'intelligence documentaire qui automatise l'ingestion, l'indexation sémantique, l'interrogation en langage naturel et la production de réponses traçables à partir de documents d'entreprise, tout en intégrant des garanties mesurables de pertinence et des dispositions de déploiement compatibles avec les exigences industrielles de sécurité et de gouvernance.

1.4.2 Objectifs spécifiques

Pour atteindre cet objectif général, le projet se décline en objectifs spécifiques mesurables :

- Automatiser l'ingestion et le prétraitement des documents (OCR, normalisation, découpage en segments) pour produire des unités d'indexation robustes.
- Mettre en place une indexation vectorielle et une stratégie de recherche sémantique incluant des critères de confiance : définir et documenter des seuils de pertinence et des métriques (τ , marges relatives, scores calibrés) pour décider du cas « aucune information trouvée ».
- Garantir la traçabilité des réponses en affichant systématiquement les sources et extraits ayant servi à la génération, et en enregistrant la provenance pour audit.
- Développer un agent de dialogue en langage naturel qui ne génère des réponses qu'à partir du contenu indexé, avec mécanismes de refus ou de clarification lorsque le seuil de confiance n'est pas atteint.
- Concevoir une interface utilisateur complète (chatbot, recherche sémantique autonome, tableau de bord administrateur) facilitant la consultation, le filtrage et le suivi d'usage.
- Définir et prototyper des options de déploiement (poste local prototype, environnement de test contrôlé, architecture cible entreprise) en précisant les mesures de sécurité, de gestion des accès et de conformité sans présenter l'exécution locale comme une garantie absolue.
- Évaluer la solution sur des critères quantitatifs (précision, rappel, taux de refus pour faibles scores, latence) et qualitatifs (satisfaction utilisateur, pertinence perçue) dans des scénarios métier représentatifs.

1.4.3 Solution proposée

Pour répondre à ces limites, nous proposons **Clarté**, une application web permettant :

- L’import de documents d’entreprise (PDF, DOCX, TXT) ;
- Leur découpage automatique en segments (chunks) et leur indexation sémantique dans une base vectorielle ;
- L’interrogation en langage naturel, via un chat bot, avec génération de réponses **exclusivement fondées sur le contenu des documents importés** ;
- L’affichage systématique des sources ayant permis de générer chaque réponse ;
- Une recherche sémantique autonome, indépendante du chat, pour consulter directement les passages pertinents ;
- Un tableau de bord administrateur pour le suivi de l’usage et la gestion des comptes utilisateurs.

À la différence des solutions RAG reposant sur des API LLM externes (OpenAI, Anthropic, etc.), le prototype Clarté exécute Ollama sur un poste local. Cette configuration limite l’exposition des documents à des services tiers, mais ne garantit pas à elle seule leur confidentialité : elle dépend aussi des accès, des journaux et des flux réseau. En production, le déploiement doit s’effectuer sur une infrastructure maîtrisée, avec contrôle d’accès chiffrement et supervision.

1.5 Choix du modèle de développement

1.5.1 Méthodologie du travail

La méthodologie adoptée pour le projet Clarté suit une approche rigoureuse et structurée afin de garantir une planification optimale, une exécution efficace et une livraison en phase avec les attentes des utilisateurs. Elle a débuté par une analyse des besoins fonctionnels et techniques, ainsi qu’une étude des pratiques actuelles de gestion documentaire au sein de l’entreprise, afin de définir précisément les spécifications du système.

Par la suite, une conception détaillée de l’application a été réalisée, incluant la modélisation des données, l’architecture logicielle du pipeline RAG et les interfaces utilisateur, pour assurer une expérience claire et intuitive. Le développement s’est appuyé sur le framework FastAPI (Python) pour le backend et sur LangChain pour l’orchestration du pipeline RAG, tandis que React (Vite) a été utilisé pour concevoir un frontend moderne et réactif, avec un affichage des réponses en flux (streaming).

La persistance des données a été assurée par SQLite via l’ORM SQLAlchemy pour les données relationnelles (utilisateurs, documents, conversations), et par ChromaDB pour la base vectorielle. Le recours à un modèle de langage exécuté localement (Ollama) a permis

d'intégrer la génération de réponses tout en respectant la contrainte de confidentialité des documents d'entreprise.

Cette organisation méthodologique a permis d'avancer de manière structurée, tout en conservant une flexibilité suffisante pour ajuster les priorités et enrichir progressivement les fonctionnalités de Clarté au fil de l'avancement du projet.

1.5.2 Méthodologie Agile pour Clarté

La méthodologie Agile est une approche de gestion de projet qui privilégie la flexibilité, la collaboration et l'amélioration continue. Contrairement aux méthodes traditionnelles (type cascade), Agile permet de livrer progressivement des fonctionnalités et de s'adapter rapidement aux besoins qui se précisent au fil du développement.

Principes clés appliqués à Clarté :

1. Livraison incrémentale :

- Chaque fonctionnalité (authentification, import et indexation des documents, recherche sémantique, génération de réponses RAG, historique des conversations, tableau de bord administrateur) a été développée et testée par étapes successives.
- Chaque itération a produit une application fonctionnelle de bout en bout, enrichie progressivement, plutôt qu'un périmètre figé entièrement en amont.

2. Collaboration constante :

- Un suivi régulier avec les encadrants a permis d'ajuster les priorités au fil de l'avancement.
- Les besoins exprimés (par exemple l'ajout de la gestion des documents de tous les utilisateurs côté administrateur, ou l'optimisation du temps de réponse) ont été intégrés au fur et à mesure, sans remise en cause du travail déjà réalisé.

3. Adaptabilité :

- Lorsqu'un besoin devenait prioritaire (par exemple l'affichage des réponses en flux plutôt qu'en bloc, pour réduire le temps d'attente perçu), il a pu être intégré rapidement grâce à une architecture modulaire (routeurs FastAPI indépendants par fonctionnalité, composants React isolés).
- Les modifications apportées à une fonctionnalité n'ont pas perturbé l'ensemble du système, chaque module (authentification, documents, chat, historique, administration) restant indépendant.

4. Amélioration continue :

- Chaque fonctionnalité livrée a été testée manuellement puis, pour le pipeline RAG, évaluée quantitativement (harnais d'évaluation automatisé, voir chapitre 4).

- Les résultats de ces tests ont directement motivé des ajustements ultérieurs (limitation du nombre de tokens générés, maintien du modèle de langage en mémoire entre les requêtes, correction de cas limites identifiés en cours de développement).

Avantages de cette approche pour Clarté :

- Obtention rapide d’une application fonctionnelle de bout en bout, dès les premières itérations.
- Meilleure adéquation du résultat final aux besoins réels, grâce à l’intégration continue des retours.
- Réduction des risques grâce à des tests et à une évaluation quantitative réguliers du pipeline RAG.
- Flexibilité pour intégrer des fonctionnalités non anticipées initialement (gestion des documents multi-utilisateurs côté administrateur, streaming des réponses, téléchargement de rapports par échange).

1.6 Planning prévisionnel

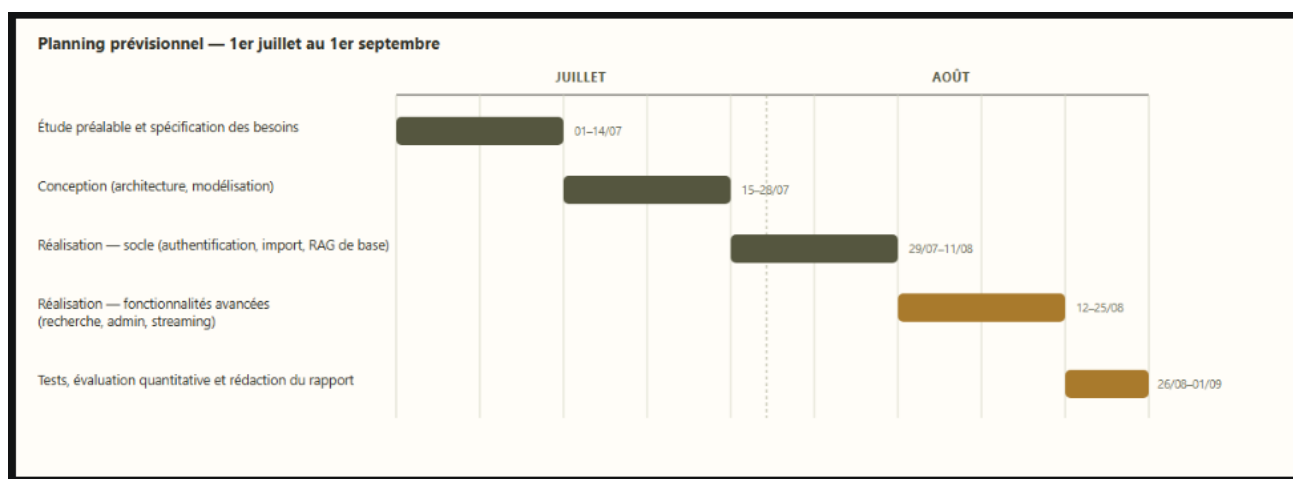


Figure 1 - Planning prévisionnel

1.7 Conclusion

Ce chapitre a présenté le contexte organisationnel du projet, l'existant observé en matière de consultation documentaire ainsi que ses principales limites. Il a également introduit la solution Clarté et le cadre méthodologique retenu pour le projet. Ces éléments constituent la base de l'analyse des exigences. Le chapitre suivant détaille les besoins fonctionnels et non fonctionnels de l'application.

Chapitre 2

Spécification des besoins

2.1 Introduction

Ce chapitre présente les besoins fonctionnels et non fonctionnels auxquels doit répondre l'application, ainsi que les principaux cas d'utilisation identifiés. Avant d'entrer dans le détail de ces besoins, il est nécessaire de revenir sur le contexte qui a motivé le projet : quel problème cherche-t-on à résoudre, et pour qui ?

Contexte et problématique

L'organisme d'accueil l'OCP Group, manipule au quotidien un grand nombre de documents internes — politiques, procédures, notes de service — dont certains présentent un caractère sensible et ne peuvent, à ce titre, être communiqués à l'extérieur de l'entreprise sans précaution.

2.2 Spécification des besoins fonctionne

Ce sont l'ensemble des fonctionnalités que le système doit offrir, organisées par thème. Le tableau 1 en donne une vue synthétique ; chaque thème est ensuite détaillé et, lorsque cela est pertinent, illustré par un cas d'usage concret.

Tableau 1 - Synthèse des besoins fonctionnels

Thème	Description synthétique
Authentification	Inscription, connexion sécurisée par jeton JWT, mots de passe jamais stockés en clair
Documents	Import de fichiers PDF/DOCX/TXT (5 Mo max), consultation et suppression
Indexation	Découpage en segments, génération d'embeddings, stockage vectoriel avec référence au document source
Recherche et génération	Recherche sémantique, génération de réponses sourcées par un LLM local, affichage en streaming, rapport téléchargeable
Historique	Sauvegarde, consultation et suppression des conversations
Administration	Tableau de bord, gestion des comptes et de l'ensemble des documents, avec garde-fous de sécurité

2.2.1 Authentification et gestion des comptes

Un utilisateur peut créer un compte à l'aide de son adresse email et d'un mot de passe, puis se connecter pour obtenir un jeton d'authentification (JWT) valable pour une durée configurable. Ce jeton est joint à chaque requête protégée, ce qui évite de retransmettre le mot de passe à chaque fois ; celui-ci n'est par ailleurs jamais stocké en clair, mais sous forme de hachage (bcrypt).

Exemple d'usage : un nouvel utilisateur crée un compte avec son email, se connecte, et accède immédiatement à l'espace de chat sans démarche supplémentaire.

2.2.2 Import et gestion des documents

Un utilisateur peut importer un document au format PDF, DOCX ou TXT, dans la limite de 5 Mo par fichier. Il peut ensuite consulter la liste de ses documents — avec le nombre de segments indexés et la date d'import — et en supprimer un, ce qui entraîne également la suppression de ses données vectorielles associées.

Exemple d'usage : un collaborateur importe la politique de congés de son service afin de pouvoir l'interroger directement, sans avoir à la relire intégralement.

2.2.3 Extraction et indexation des documents

Dès son import, le texte d'un document est extrait puis découpé automatiquement en segments (chunks) de taille configurable. Chaque segment est converti en vecteur numérique (embedding) et stocké dans une base vectorielle, avec une référence à son document d'origine, ce qui permet de retrouver précisément la source de chaque information restituée par la suite.

2.2.4 Recherche sémantique

Indépendamment du chat, un utilisateur peut interroger directement sa base documentaire depuis une page « Recherche » et obtenir les passages les plus pertinents par rapport à sa requête. Cette recherche reste strictement limitée aux documents lui appartenant.

Exemple d'usage : un utilisateur retrouve rapidement le passage exact d'une procédure mentionnant un délai précis, sans avoir à reformuler sa demande sous forme de question

2.2.5 Génération de réponses (chat RAG)

Un utilisateur peut poser une question en langage naturel ; le système récupère les $k = 2$ passages les plus proches et calcule leur score de similarité normalisé (compris entre 0 et 1). Si le meilleur score est inférieur à 0,65, les passages sont considérés comme insuffisamment pertinents : le système indique alors clairement qu'aucune information n'a été trouvée, sans appeler le modèle de langage. Sinon, seuls les passages validés sont transmis au modèle et la réponse s'appuie exclusivement sur ce contenu. Le seuil de 0,65 est calibré sur un jeu de questions annotées et doit être réévalué en cas de changement du corpus ou du modèle d'embeddings

Exemple d'usage : un utilisateur importe une politique RH puis demande combien de jours de congés il a, et obtient la réponse exacte, accompagnée de la source citée.

2.2.6 Historique des conversations

Chaque échange (question, réponse, sources) est sauvegardé et rattaché à une conversation et à un utilisateur. Celui-ci peut à tout moment consulter, reprendre ou supprimer une conversation existante.

2.2.7 Administration

Un compte administrateur dispose d'un tableau de bord dédié, distinct de l'espace utilisateur standard, présentant des statistiques globales d'usage (utilisateurs, documents, segments indexés, conversations, messages, volume de stockage). Il peut créer, modifier ou supprimer un compte utilisateur, ainsi que consulter et supprimer n'importe quel document importé par n'importe quel utilisateur — des garde-fous empêchant toutefois un administrateur de se retirer ses propres droits ou de supprimer son propre compte. Un utilisateur non-administrateur n'a accès à aucune de ces fonctionnalités.

Exemple d'usage : un administrateur constate, via le tableau de bord, qu'un document volumineux occupe une part importante du stockage et le supprime directement depuis l'interface, sans intervention technique.

2.3 Spécification des besoins non fonctionnels

Ce sont les contraintes qui ne sont pas, à proprement parler, des fonctionnalités, mais qui conditionnent la qualité et l’acceptabilité du système. Le tableau 2.2 en donne une vue synthétique ; chacune est ensuite justifiée par rapport au contexte métier décrit en introduction de ce chapitre.

Tableau 2 - Synthèse des besoins non fonctionnels

Besoin	Détail
Confidentialité	Prototype local, test contrôle, production sécurisée
Isolation	Chaque utilisateur ne voit que ses propres documents
Sécurité	Mots de passe hachés, jetons signés, séparation des Privilèges
Performance	Réponses en streaming, génération plafonnée en Tokens.
Ergonomie	Interface claire, accessible au clavier
Portabilité	Application web, aucune installation cliente requise

Confidentialité. Les documents traités sont internes, parfois sensibles : les transmettre à une API LLM externe romprait cette confidentialité. Le modèle de langage est donc exécuté localement, sans qu’aucune donnée ne quitte l’entreprise.

Isolation. Plusieurs utilisateurs partagent le même déploiement ; sans isolation, l’un pourrait recevoir une information issue du document d’un autre. Un filtrage systématique par propriétaire garantit donc que chacun ne voit que ses propres documents.

Sécurité. L’application donne accès à des documents d’entreprise : une faille d’authentification les exposerait directement. D’où le hachage des mots de passe (bcrypt), les jetons signés (JWT) et la séparation stricte des privilèges administrateur/utilisateur.

Performance. Sans GPU, un temps de réponse trop long découragerait l’usage au quotidien. La génération est donc plafonnée en tokens et affichée en streaming, pour rester utilisable malgré l’exécution locale.

Ergonomie. Les utilisateurs visés n’ont pas de compétence technique particulière : l’adoption de l’outil dépend directement de la simplicité de son interface.

Portabilité. Déployer un client sur chaque poste représenterait un coût de maintenance à l’échelle de l’OCP Group. Clarté reste donc une application web, sans installation.

2.4 Choix d'architecture — Le pipeline RAG

Définition

Le RAG (Retrieval-Augmented Generation) est une approche hybride qui combine deux capacités : d'abord, retrouver dans une base documentaire les passages les plus pertinents pour une requête donnée ; ensuite, fournir ces passages en tant que contexte à un modèle de langage qui génère la réponse. L'idée centrale est que le modèle ne doit s'appuyer que sur les preuves issues des documents (les « passages récupérés ») et non sur des connaissances externes susceptibles d'entraîner des hallucinations. En pratique, cela permet d'obtenir des réponses plus factuelles et traçables.

Phase d'indexation : étapes et justifications

Extraction du texte depuis PDF, DOCX, TXT

OCR pour les PDF scannés : utiliser un moteur OCR robuste (Tesseract, ou un service OCR local/entreprise) avec réglages de prétraitement (binarisation, correction d'inclinaison) pour maximiser la qualité du texte extrait.

Parsers natifs pour DOCX/TXT : récupérer le texte et la structure (titres, paragraphes, tableaux) pour conserver des repères sémantiques utiles aux métadonnées.

Justification : la qualité du texte d'entrée conditionne directement la qualité des embeddings et des extractions suivantes ; un bon prétraitement réduit le bruit et les faux positifs.

Normalisation et nettoyage

Suppression des artefacts, normalisation des espaces, détection de la langue, conservation des numéros et unités dans un format stable.

Justification : embeddings plus cohérents et meilleure comparabilité inter-documents.

Découpage en segments (chunks) avec chevauchement

Découper un document en segments de longueur contrôlée (par ex. 400–800 tokens / ~250–600 mots selon le modèle d'embedding), avec chevauchement de 20–30 % (p. ex. 100–200 tokens) entre segments.

Stocker pour chaque segment : identifiant (document, page, offset), texte brut et extrait nettoyé.

Justification : garder un chevauchement évite de perdre le contexte aux limites des segments (phrases coupées, informations réparties sur deux segments). Des segments trop longs pénalisent le coût des embeddings et la granularité des citations ; trop courts, ils fragmentent le sens.

Génération d'un vecteur d'embedding par segment

Choix du modèle d'embedding : arbitrer entre performance sémantique et contraintes (coût, exécution locale vs API). Pour un prototype local, privilégier un modèle embarqué compatible avec la contrainte de confidentialité ; en production, choisir un modèle calibré sur la langue et le domaine.

Normalisation des vecteurs (L2 / cosinus) et indexation dans la base vectorielle.

CHAPITRE 2. SPÉCIFICATION DES BESOINS

Justification : les embeddings traduisent le sens textuel en représentation mathématique et sont au cœur de la recherche par similarité.

Stockage dans ChromaDB avec métadonnées

Stocker : embedding, texte du segment, métadonnées (propriétaire, nom du document, page, horodatage, checksum, langue, tags métier).

Activer le filtrage par métadonnée (p. ex. owner) pour isoler les recherches par client ou service.

Justification : ChromaDB offre une solution légère, locale et performante pour stocker embeddings et métadonnées ; le support de filtres permet de réaliser des recherches multi-tenantes et conformes aux règles d'accès. Les métadonnées facilitent aussi la traçabilité et l'audit.

Phase de question / réponse — étapes et justifications

1. Transformation de la question en embedding
 - Prétraitement de la requête (normalisation, détection de langue, expansion si nécessaire) puis génération d'un embedding avec le même modèle que celui utilisé pour les segments. Justification : cohérence des espaces d'embeddings maximise la qualité du voisinage.
2. Recherche des k segments les plus proches dans ChromaDB (filtrés par propriétaire)
 - Exécuter une recherche top-k (par ex. $k = 8-15$) en utilisant la similarité cosinus ; appliquer un filtre metadata owner pour ne retourner que les segments autorisés.
 - Optionnel : combiner retrieval dense (embeddings) avec un reranking BM25/lexical pour couvrir les cas où la similarité sémantique échoue. Justification : filtrage par métadonnées assure l'isolement des corpus multi-tenants ; la combinaison dense+ lexical améliore la robustesse sur documents très techniques.
3. Sélection et réclassement / application de seuils de pertinence
 - Calculer un score de confiance (meilleur score, marge avec le deuxième score, moyenne pondérée) et le comparer à des seuils τ définis.
 - Si aucun segment n'atteint τ ou si la marge est insuffisante, déclencher une politique de refus/clarification (« Aucune information pertinente trouvée ») plutôt que de générer une réponse incertaine. Justification : éviter les faux positifs et les réponses non fondées ceci met en œuvre la règle mesurable de « absence d'information » et améliore la fiabilité.
4. Construction du contexte (segments + question)
 - Concaténer les segments sélectionnés (avec citations claires : document, page, segment id) en respectant la limite de tokens du LLM ; éventuellement résumer ou prioriser si la fenêtre-context est limitée. Justification : fournir au LLM un contexte restreint et pertinent permet de l'enfermer dans les preuves documentaires.
5. Génération de la réponse par le LLM en streaming
 - Utiliser un prompt système strict demandant explicitement au modèle de n'utiliser que le

contexte fourni et de citer les extraits utilisés.

- Diffuser la réponse en streaming côté client pour réduire la latence perçue et permettre un affichage progressif des sources.
Justification : le streaming améliore l'UX ; un prompt contraignant et la fourniture unique du contexte réduisent les risques d'hallucination.
- 6. Sauvegarde de l'échange dans l'historique
 - Enregistrer : requête utilisateur, top segments consultés (ids + scores), contexte fourni, réponse générée, durée, utilisateur, métadonnées de confidentialité, et un indicateur de confiance.
 - Conserver les preuves (extraits cités) pour audit et possibilité de rétro-ingénierie ou d'entraînement de modèles de re-ranking.
Justification : l'historique permet l'audit, le traçage des sources, l'analyse des erreurs et l'amélioration continue (calibration des seuils, affinement des prompts).

2.5 Présentation des cas d'utilisation

2.5.1 Présentation des acteurs

L'application distingue deux acteurs, aux besoins et aux espaces d'utilisation entièrement séparés : l'utilisateur, qui exploite le chatbot au quotidien, et l'administrateur, qui supervise la plateforme sans jamais accéder à l'espace de chat.

Utilisateur : C'est un collaborateur de l'entreprise, sans compétence technique particulière requise. Après inscription, il importe ses propres documents, leur pose des questions en langage naturel via le chat, consulte son historique de conversations, effectue des recherches sémantiques indépendantes du chat, et peut télécharger un rapport pour chaque réponse obtenue. Il n'a accès qu'à ses propres documents et conversations.

Administrateur : Ce rôle est attribué à certains comptes via une liste d'emails autorisés configurée par l'entreprise ; un compte administrateur est automatiquement redirigé vers un tableau de bord dédié dès la connexion, et n'utilise jamais l'espace de chat. L'administrateur y consulte des statistiques globales d'usage, gère l'ensemble des comptes utilisateurs (création, modification, suppression) et peut consulter ou supprimer n'importe quel document importé par n'importe quel utilisateur.

Tableau 3 - Synthèse des acteurs

	Utilisateur	Administrateur
Accès	Chat, recherche, historique	Tableau de bord uniquement
Périmètre de données	Ses propres documents et conversations	L'ensemble des utilisateurs et documents
Attribution du rôle	Par défaut, à l'inscription	Liste d'emails autorisés

2.5.2 Description des cas d'utilisation

Les fonctions de l'Utilisateur et de l'Administrateur étant complètement indépendantes aucune fonctionnalité de Clarté ne fait intervenir les deux acteurs simultanément les cas

CHAPITRE 2. SPÉCIFICATION DES BESOINS

d'utilisation sont présentés par acteur. Les cas simples, sans ambiguïté de déroulement, sont listés brièvement ci-dessous ; les deux cas dont le scénario mérite d'être détaillé (recours à un traitement particulier, ou risque de confusion sur le comportement attendu) sont décrits par une fiche complète .

Cas d'utilisation de l'acteur Utilisateur : S'inscrire, Se connecter, Importer un document, Supprimer un document, Rechercher dans les documents, *Poser une question* (détaillé ci-après), Consulter l'historique, Télécharger un rapport de réponse.

Cas n°	1
Acteur(s)	Utilisateur
Objectif	Obtenir une réponse fiable et sourcée à une question portant sur ses documents importés.
Pré-condition(s)	L'utilisateur est authentifié et a importé au moins un document.
Post-condition(s)	Une réponse est affichée, accompagnée des sources documentaires utilisées ; l'échange est sauvegardé dans l'historique.
Scénario nominal	1. L'utilisateur saisit sa question. 2. Le système recherche les passages pertinents. 3. Le système transmet la question et le contexte au LLM. 4. La réponse est générée et affichée progressivement. 5. L'échange est enregistré.
Scénario alternatif	Si aucun passage pertinent n'est trouvé, le système informe l'utilisateur qu'aucune information n'est disponible plutôt que de générer une réponse.

Tableau 4 - Description du cas d'utilisation « Poser une question » pour l'acteur Utilisateur

Cas d'utilisation de l'acteur Administrateur : Se connecter, Consulter le tableau de bord, *Gérer les utilisateurs et les documents* (détaillé ci-après).

Tableau 5 - Description du cas d'utilisation « Gérer les utilisateurs et les documents » pour l'acteur Administrateur

Cas n°	2
Acteur(s)	Administrateur
Objectif	Créer, modifier ou supprimer un compte utilisateur, et consulter ou supprimer un document appartenant à n'importe quel utilisateur, depuis le tableau de bord.
Pré-condition(s)	L'administrateur est authentifié et reconnu comme tel par le système.
Post-condition(s)	Le compte ou le document est créé, mis à jour ou supprimé ; en cas de suppression d'un utilisateur ou d'un document, les données vectorielles associées sont également supprimées.
Scénario nominal	1. L'administrateur accède au tableau de bord. 2. Il consulte la liste des utilisateurs et/ou des documents. 3. Il crée/modifie/supprime un compte, ou supprime un document. 4. Le tableau de bord est actualisé.
Scénario alternatif	Le système refuse toute tentative de l'administrateur de supprimer ou de retirer ses propres droits d'administration.

2.5.3 Diagramme des cas d'utilisation global

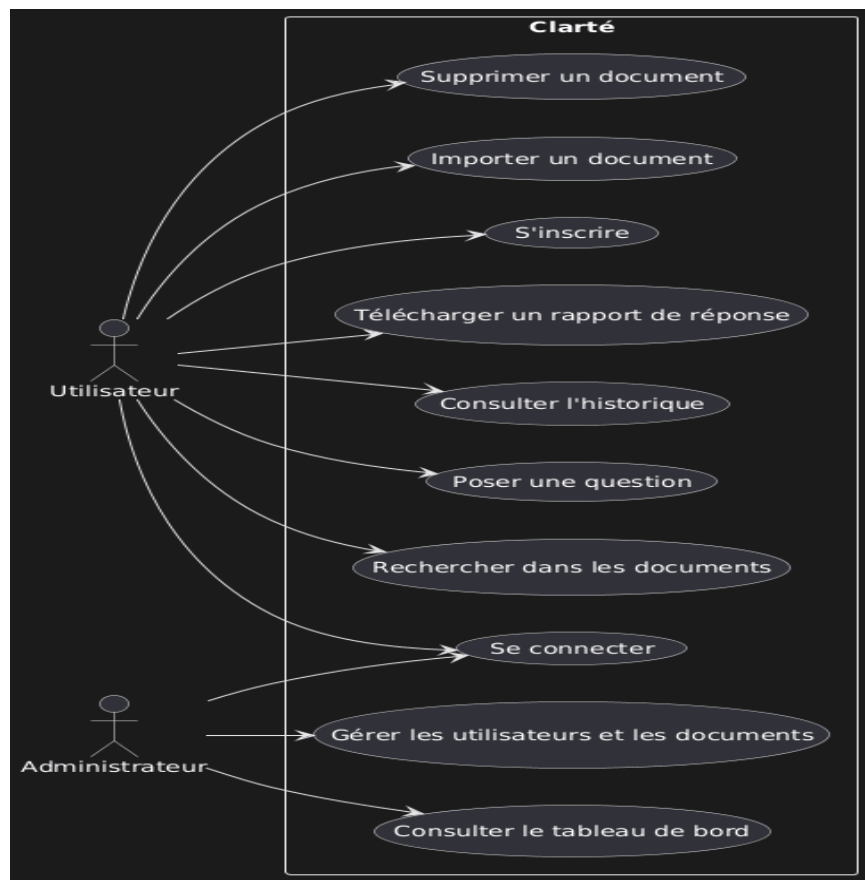


Figure 2 - Diagramme de cas d'utilisation globale

Conclusion

Ce chapitre a précisé les besoins fonctionnels et non fonctionnels de Clarté, ainsi que les principaux cas d'utilisation associés aux utilisateurs et à l'administrateur. Il a traduit les attentes métier en exigences concernant notamment la recherche, la génération de réponses, la sécurité et l'isolation des données. Le chapitre suivant présente la conception du système retenue pour répondre à ces besoins

Chapitre 3

Conception du système

3.1 Introduction

Ce chapitre présente la conception de l'application selon les deux points de vue complémentaires du langage UML : la modélisation dynamique, qui décrit le comportement du système au fil du temps, et la modélisation statique, qui décrit sa structure de données et son architecture.

3.2 Modélisation dynamique

3.2.1 Diagramme de séquence — Authentification

Le scénario d'authentification se déroule en sept étapes, numérotées sur la figure 3.2.1 : (1) l'utilisateur saisit son email et son mot de passe ; (2) le frontend transmet ces informations au backend (POST /auth/login) ; (3) le backend vérifie le mot de passe auprès de la base de données (comparaison avec le hash bcrypt stocké) ; (4) si la vérification réussit, il génère un jeton JWT signé ; (5) ce jeton est retourné au frontend. Pour chaque requête ultérieure, (6) le frontend joint ce jeton via l'en-tête Authorization, ce qui permet (7) au backend de répondre en identifiant l'utilisateur sans revalider son mot de passe.

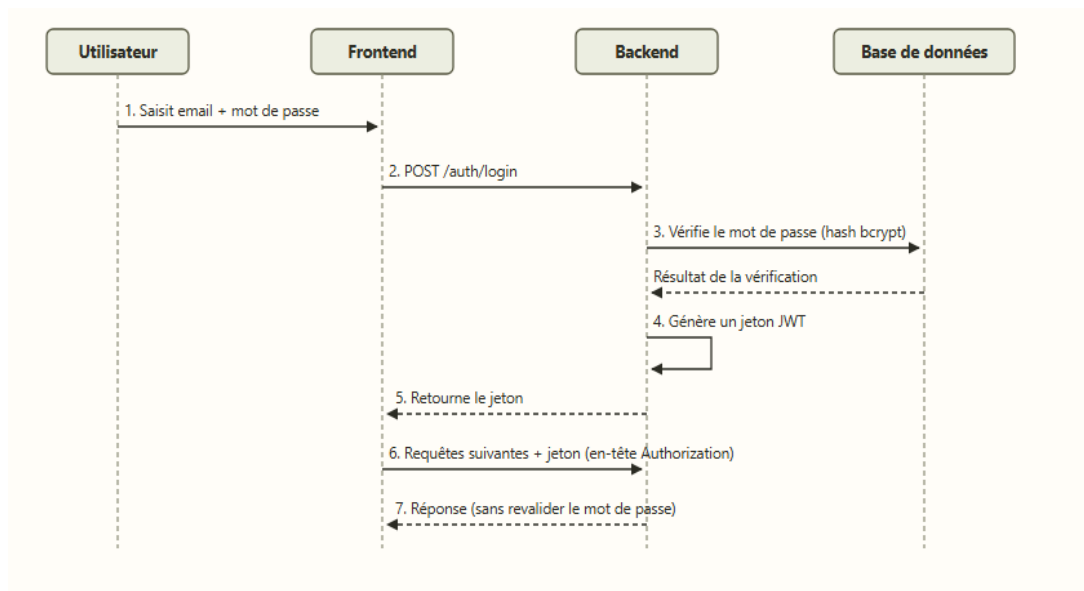


Figure 3 - Diagramme de séquence — Authentification

3.2.2 Diagramme de séquence — Question/réponse RAG

Ce scénario illustre le cœur fonctionnel de l'application ; les étapes ci-dessous reprennent exactement la numérotation de la figure 3.2 :

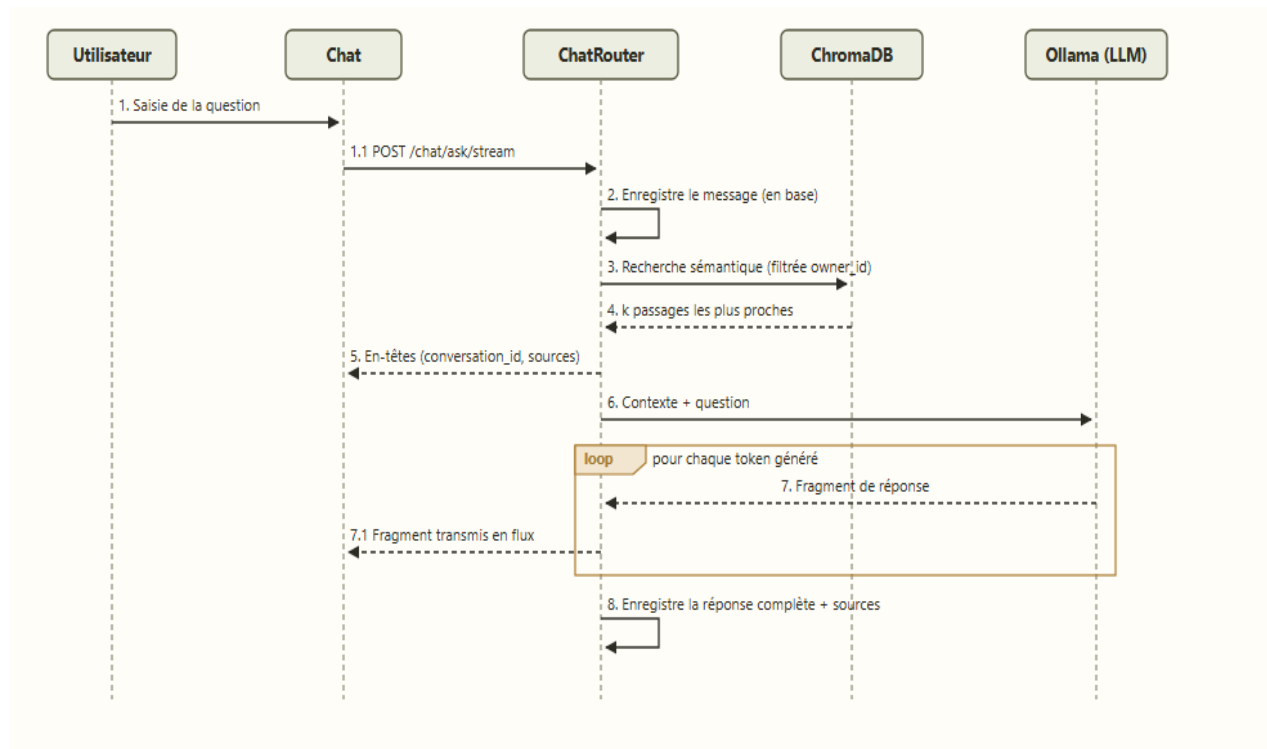


Figure 4 - Diagramme de séquence — Rag

1. L'utilisateur saisit sa question depuis l'interface de chat.
- 1.1 Le frontend transmet la question au backend (POST /chat/ask/stream).
2. Le backend enregistre le message utilisateur en base (création ou réutilisation d'une conversation).
3. Le backend effectue une recherche sémantique (`similarity_search`) dans ChromaDB, filtrée par l'identifiant du propriétaire.
4. . ChromaDB renvoie les $k = 2$ passages les plus proches, avec leurs scores de similarité.
5. Ces sources sont immédiatement connues et transmises au frontend via les en-têtes de la réponse HTTP, avant même le début de la génération.
6. Le backend transmet le contexte et la question au modèle de langage local (Ollama / Llama 3).
7. Le LLM génère la réponse **token par token** ; chaque fragment (7.1) est transmis immédiatement au frontend (streaming HTTP), qui l'affiche au fur et à mesure ces deux étapes se répètent jusqu'à la fin de la génération (bloc `loop` sur la figure).
8. Une fois la génération terminée, le backend enregistre la réponse complète avec ses sources.

Le statut « aucune information trouvée » ne dépend donc pas uniquement d'une liste vide : il est déclenché lorsque le meilleur des $k = 2$ passages obtient un score de similarité normalisé inférieur à 0,65. Ce seuil est calibré sur un jeu de validation et réévalué si le corpus ou le modèle d'embeddings change.

3.2.3 Diagramme d'activité — Import et indexation d'un document

Le diagramme d'activité ci-dessous décrit le déroulement interne du traitement d'un document, avec ses deux points de décision (validation du fichier, réussite du traitement) et ses chemins d'échec.

Après le dépôt du fichier, une vérification contrôle le format (PDF, DOCX, TXT) et la taille (≤ 5 Mo) ; en cas d'échec, la requête est rejetée. Sinon, le document passe par trois étapes (extraction du texte, découpage en segments, génération des embeddings) avant indexation dans ChromaDB avec ses métadonnées (`owner_id`, `document_id`). Si une erreur survient durant ce traitement, le document et son fichier physique sont supprimés ; sinon, le nombre de segments est mis à jour en base relationnelle et le processus se termine avec succès.

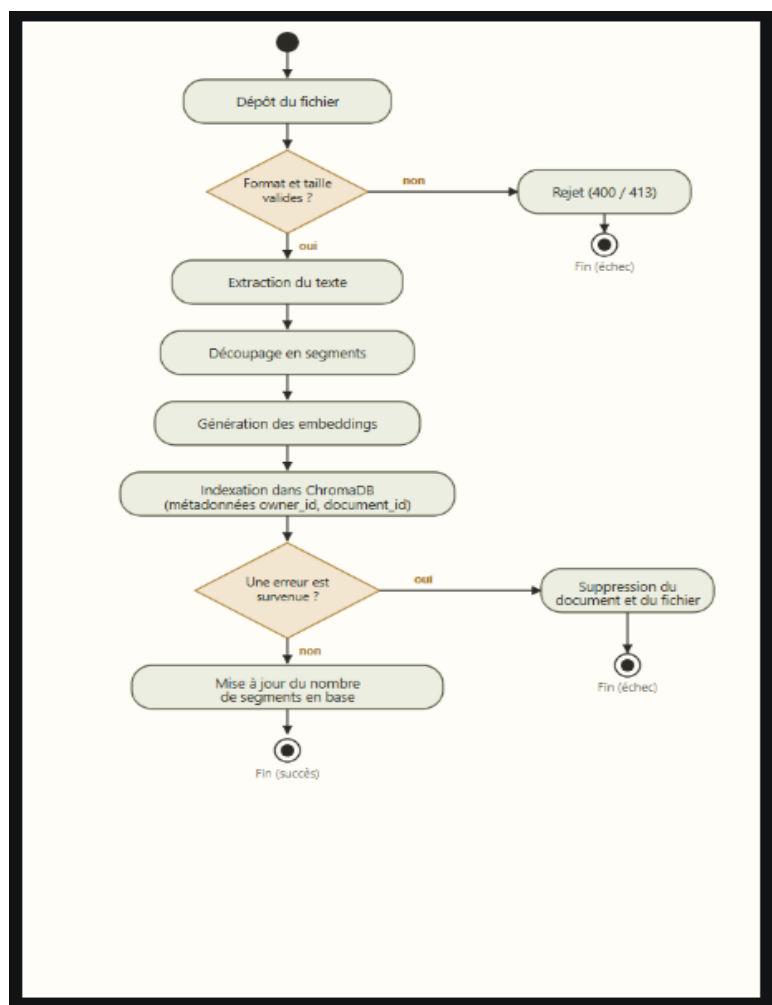


Figure 5 - Diagramme d'activité

3.3 Modélisation statique

3.3.1 Diagramme de classes

Le modèle de données relationnel s’articule autour de quatre entités principales :

- **User** : représente un compte (utilisateur ou administrateur).
- **Document** : représente un fichier importé par un utilisateur.
- **Conversation** : regroupe une suite d’échanges entre un utilisateur et l’assistant.
- **Message** : représente un échange individuel (question ou réponse) au sein d’une conversation.

Relations : un User possède plusieurs Document et plusieurs Conversation (relations 1,n) ; une Conversation contient plusieurs Message (relation 1,n). La suppression d’un utilisateur ou d’une conversation entraîne la suppression en cascade des entités qui lui sont rattachées.

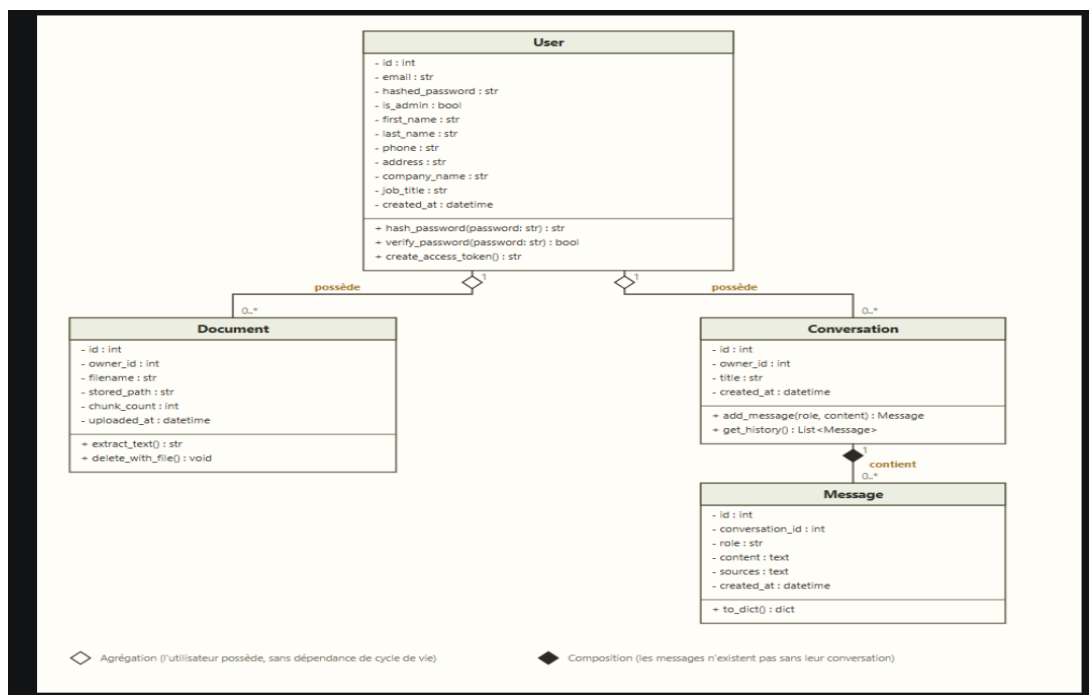


Figure 6 - Diagramme de classes

3.3.2 Modèle relationnel

La dérivation du diagramme de classes vers le modèle relationnel est directe : chaque classe devient une relation, chaque attribut devient une colonne, et chaque association devient une clé étrangère portée par la relation du côté « plusieurs ». Le schéma obtenu est le suivant (clé primaire soulignée, # pour une clé étrangère) :

User (id, email, hashed_password, is_admin, created_at, first_name, last_name, adresse, company_name, phone, job_title)

Document (id, owner_id#, filename, stored_path, chunk_count, uploaded_at)

Conversation (id, owner_id#, title, created_at)

Message (id, conversation_id#, role, content, sources, created_at)

Les clés étrangères référencent les relations suivantes :

- Document.owner_id → User.id
- Conversation.owner_id → User.id
- Message.conversation_id → Conversation.id

Ce schéma respecte la troisième forme normale (3FN) : chaque attribut non clé dépend uniquement de la clé primaire de sa relation, sans dépendance transitive ni redondance. Le détail de chaque colonne (type, obligation, valeur par défaut) est donné dans le dictionnaire de données ci-dessous.

3.3.3 Dictionnaire de données

Tableau 6 - Dictionnaire de données - table users

Colonne	Type	Obligatoire	Défaut	Clé
id	Entier	Oui	auto-incrément	Primaire
email	Texte	Oui	—	Unique
hashed_password	Texte	Oui	—	—
is_admin	Booléen	Oui	Faux	—
created_at	Date/heure	Non	horodatage courant	—
first_name	Texte	Oui	—	—
last_name	Texte	Oui	—	—
phone	Texte	Oui	—	—
address	Texte	Oui	—	—
company_name	Texte	Oui	—	—
job_title	Texte	Oui	—	—

Tableau 7 - Dictionnaire de données — table documents

Colonne	Type	Obligatoire	Clé
id	Entier	Oui	Primaire
owner_id	Entier	Oui	Étrangère → users.id
filename	Texte	Oui	—
stored_path	Texte	Oui	—
chunk_count	Entier	Non (déf. 0)	—
uploaded_at	Date/heure	Non	—

Tableau 8 - Dictionnaire de données - table conversations

Colonne	Type	Obligatoire	Clé
id	Entier	Oui	Primaire
owner_id	Entier	Oui	Étrangère → users.id
title	Texte	Non	—
created_at	Date/heure	Non	—

Tableau 9 - Dictionnaire de données - table messages

Colonne	Type	Obligatoire	Clé
id	Entier	Oui	Primaire
conversation_id	Entier	Oui	Étrangère → conversations.id
role	Texte (« user »/« assistant »)	Oui	—
content	Texte long	Oui	—
sources	Texte (JSON)	Non	—
created_at	Date/heure	Non	—

3.3.4 Architecture de l'application

Architecture logicielle

L'application suit une architecture **client-serveur découplée** :

- **Frontend** : SPA en **React** (Vite), communiquant avec le backend par API REST et flux HTTP (streaming) pour les réponses du chatbot.
- **Backend** : API **Python/FastAPI**, orchestrant l'authentification (JWT, bcrypt), le pipeline RAG via **LangChain**, la persistance relationnelle via **SQLAlchemy** (SQLite), la base vectorielle **ChromaDB**, et la génération de texte via **Ollama** (Llama 3, en local).
- **Séparation stricte** entre interface utilisateur et interface d'administration.

Le diagramme de composants situe ces éléments : le frontend ne dialogue jamais directement avec ChromaDB, SQLite ou Ollama, toutes les communications transitant par le backend, point d'entrée unique garantissant authentification et isolation par `owner_id`.

Le pipeline RAG comporte deux phases qui se rejoignent au niveau de ChromaDB : l'indexation y **écrit** (import → découpage → embeddings), la question/réponse y **lit** (embedding → recherche filtrée par `owner_id` → contexte → LLM). Ce filtre par propriétaire garantit l'isolation des données entre comptes .

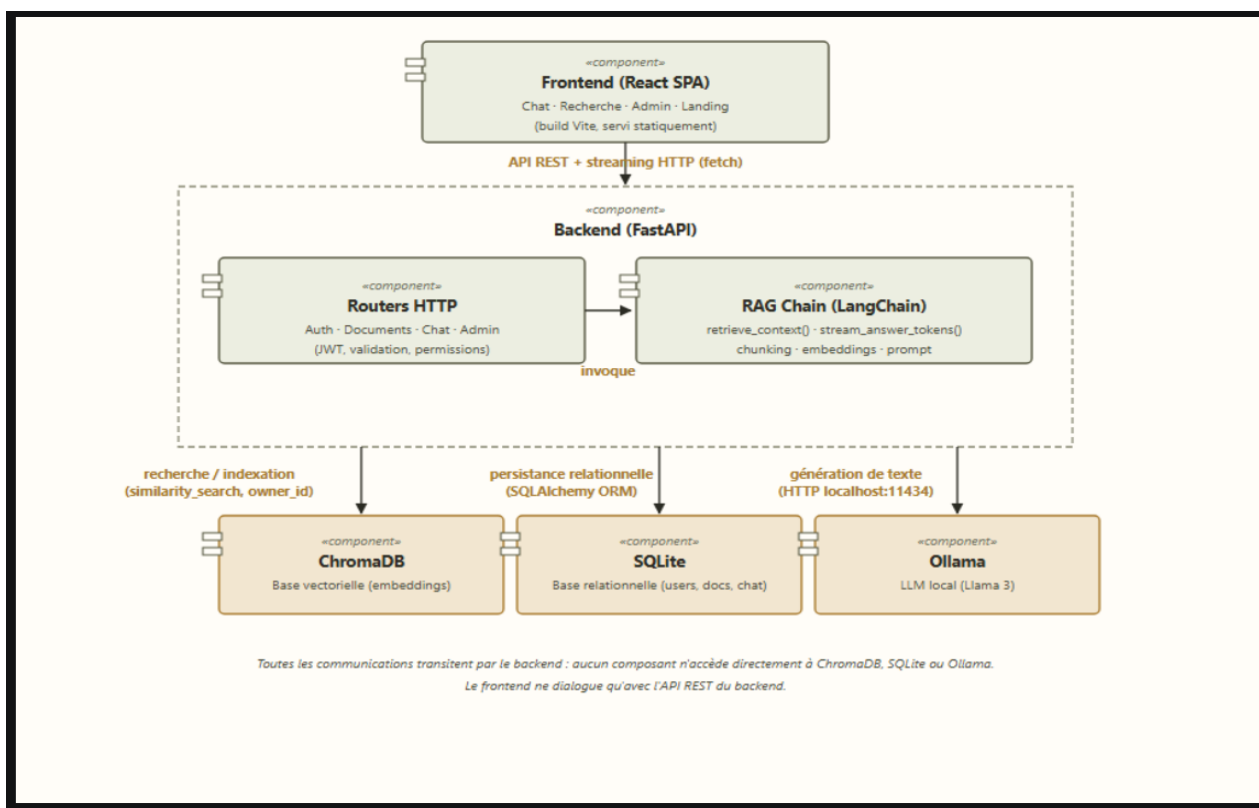


Figure 7 - Diagramme de composant de l'architecture logicielle de Clarté

3.3.5 Architecture matérielle

Dans le cadre du projet, l'application est déployée sur un poste de travail unique. Le diagramme de déploiement distingue deux nœuds : le poste client, exécutant uniquement le navigateur, et le poste hébergeant le backend, les bases de données et le LLM, reliés en HTTP/HTTPS via le réseau local.

En production, ces composants seraient répartis sur des nœuds distincts (serveur d'application, serveur de base de données, serveur d'inférence GPU), communiquant sans exposition à internet pour préserver la confidentialité des documents.

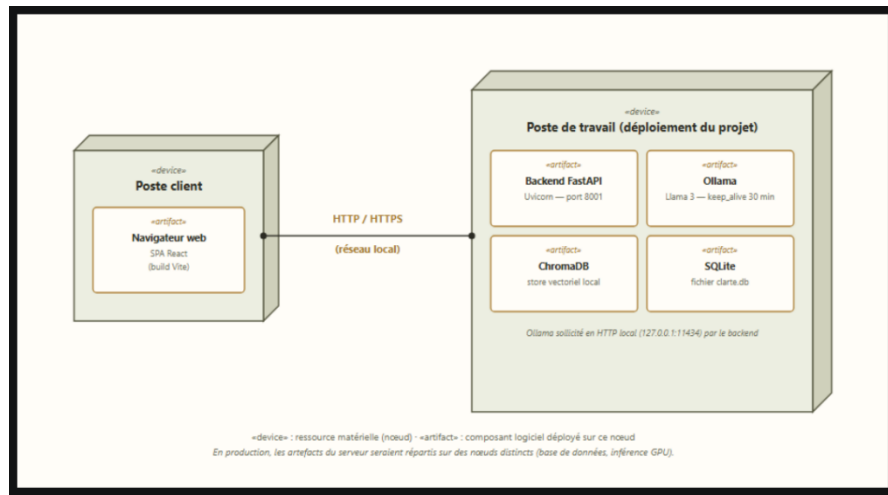


Figure 8 - Diagramme de déploiement

3.4 Conclusion

Ce chapitre a formalisé la conception de Clarté à travers la modélisation dynamique des principaux scénarios et la modélisation statique de l'architecture et des données. Il a ainsi décrit les interactions entre les composants du pipeline RAG et l'organisation technique de l'application. Le chapitre suivant présente la réalisation concrète de cette concept.

Chapitre 4

Réalisation du système

4.1 Introduction

Après avoir spécifié les besoins puis conçu l'architecture de Clarté, ce chapitre présente la concrétisation de cette conception en un produit fini. Il se compose de deux parties : la première détaille l'environnement de développement, matériel et logiciel, dans lequel l'application a été réalisée ; la seconde présente la mise en œuvre de la solution à travers ses principales interfaces graphiques.

4.2 Environnement de développement

4.2.1 Environnement matériel

Le développement a été réalisé sur un poste de travail unique, sans serveur ni infrastructure réseau dédiée : le stage s'étant déroulé à distance, l'accès aux outils s'est fait via une simple connexion internet, sans routeur, hub ou serveur spécifique à configurer côté projet. Les caractéristiques du poste utilisé sont les suivantes :

- Processeur : AMD Ryzen 5 PRO 4650U (6 cœurs / 12 threads, fréquence de base 2,1GHz)
- Mémoire vive : 16 Go installés (dont environ 14 Go utilisables par le système, une partie étant réservée au GPU intégré)
- Système d'exploitation : Windows 10 Professionnel (64 bits)
- Stockage : disque de 476 Go
- Accélération GPU : aucune carte graphique dédiée ; le modèle de langage (Llama3) est exécuté en inférence CPU via Ollama, ce qui explique en partie les temps de réponse observés lors de l'évaluation quantitative

4.2.2 Environnement logiciel

Tableau 10 - Environnement logiciel

Composant	Technologie
Langage backend	Python 3.13
Framework API	FastAPI
Framework RAG	LangChain
Base vectorielle	ChromaDB
Modèle d'embeddings	Sentence Transformers (all-MiniLM-L6-v2)
Modèle de langage (LLM)	Llama 3 (via Ollama, exécution locale)
Base relationnelle	SQLite (via SQLAlchemy)
Frontend	React 18 (Vite)
Authentification	JWT (python-jose), bcrypt (passlib)
Éditeur / outils	Visual Studio Code, Git

Backend

FastAPI (python)



Figure 9: FastAPI

FastAPI est un framework Python moderne dédié à la création d'API web performantes. Il repose sur les annotations de type Python et génère automatiquement une documentation interactive (Swagger/OpenAPI). Il est reconnu pour sa rapidité d'exécution et sa facilité de prise en main.

Framework RAG



Figure 10: LangChain

LangChain

LangChain est un framework open-source permettant de construire des applications basées sur des grands modèles de langage (LLM). Il fournit des outils prêts à l'emploi pour orchestrer des pipelines RAG, gérer les prompts, les mémoires et les connexions à des sources de données externes. Il facilite ainsi l'intégration de LLM dans des applications complexes.

Base vectorielle



Figure 11:ChromaDB

ChromaDB

ChromaDB est une base de données vectorielle open-source conçue pour stocker et rechercher des embeddings. Elle permet d'effectuer des recherches par similarité sémantique, essentielles aux systèmes de type RAG. Sa légèreté et sa simplicité d'intégration en font un choix populaire pour les projets d'IA.

Modèle d'embeddings



Figure 12: Sentence Transformers

Sentence Transformers (all-MiniLM-L6-v2)

Sentence Transformers est une bibliothèque Python permettant de transformer des phrases en vecteurs numériques (embeddings) capturant leur sens. Le modèle all-MiniLM-L6-v2 est une version légère et rapide, offrant un bon compromis entre performance et rapidité d'exécution. Il est largement utilisé pour la recherche sémantique et les systèmes RAG.

Modèle de langage (LLM)



Figure 13:Llama 3

Llama 3 (via Ollama, prototype locale)

Llama 3 est un grand modèle de langage open-source développé par Meta, conçu pour la génération de texte en langage naturel. Ollama permet de l'exécuter localement pour le prototype, ce qui limite l'exposition des données sans constituer une garantie absolue de confidentialité. Dans l'environnement de test, les accès, les journaux et les flux réseau doivent être contrôlés. L'architecture cible de l'entreprise devra s'appuyer sur une infrastructure administrée, avec chiffrement, contrôle d'accès et supervision.

Base relationnelle



Figure 14: SQLite

SQLite (via SQLAlchemy)

SQLite est un système de gestion de base de données relationnelle léger, ne nécessitant aucun serveur dédié, stockant les données dans un simple fichier. SQLAlchemy est un ORM Python qui permet de manipuler cette base à l'aide d'objets Python plutôt que de requêtes SQL brutes. Cette association simplifie le développement tout en assurant une gestion fiable des données.

Frontend

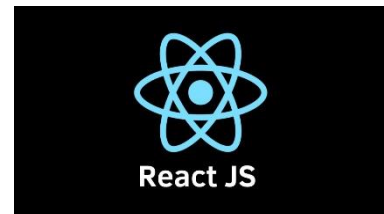


Figure 15: React.js

React 18 (Vite)

React est une bibliothèque JavaScript développée par Meta, dédiée à la construction d'interfaces utilisateur interactives et réactives. Vite est un outil de build moderne offrant un démarrage et un rechargement à chaud extrêmement rapides pendant le développement. Ensemble, ils permettent de développer des applications frontend performantes et modulaires.

Authentification



Figure 16: Bcrypt



Figure 17: JWT

JWT (python-jose) / bcrypt (passlib)

JWT (JSON Web Token) est un standard permettant de générer des jetons sécurisés pour authentifier les utilisateurs sans session côté serveur. bcrypt est un algorithme de hachage utilisé pour stocker les mots de passe de façon sécurisée, rendant leur récupération quasiment impossible. Ensemble, ces outils assurent une authentification robuste et une protection efficace des données sensibles

Éditeur / outils



Figure 19: Visual Studio



Figure 18: Git

Visual Studio Code / Git

Visual Studio Code est un éditeur de code source léger et extensible, largement utilisé pour le développement logiciel grâce à son riche écosystème d'extensions. Git est un système de contrôle de version permettant de suivre l'évolution du code source et de faciliter le travail collaboratif. Ensemble, ils constituent des outils incontournables du quotidien d'un développeur.

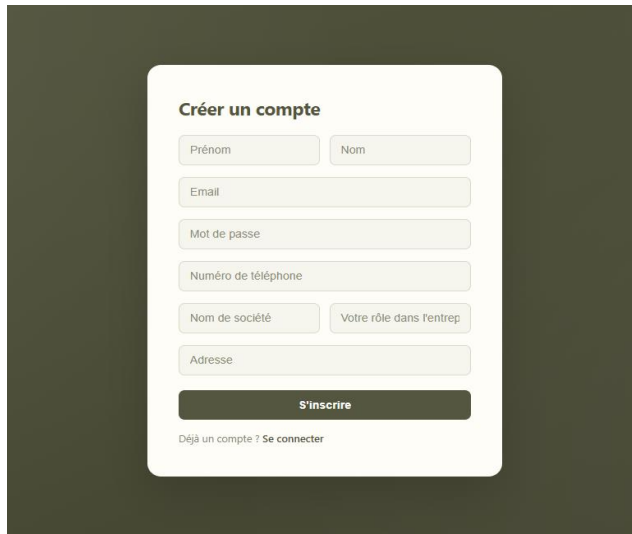
4.3 Principales interfaces graphiques

— **Page d'accueil** : première page accessible aux visiteurs non connectés, elle présente Clarté en langage accessible, sans jargon technique. Un guide en 5 étapes explicite le parcours, de l'import d'un document à l'obtention d'une réponse sourcée, avec des boutons visibles vers la connexion et l'inscription.



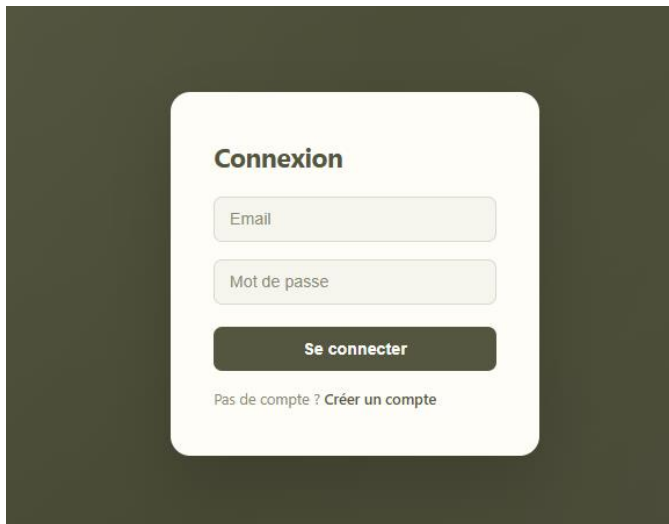
Figure 20: Page d'accueil

— **Connexion / Inscription** : formulaires classiques d'authentification et de création de compte, avec gestion explicite des erreurs (email déjà utilisé, identifiants invalides, champs manquants). Selon son rôle (is_admin), l'utilisateur est automatiquement redirigé vers l'espace utilisateur ou administrateur.



The screenshot shows a registration form titled "Créer un compte" on a dark background. The form is a light beige rounded rectangle containing several input fields: "Prénom" and "Nom" (small), "Email", "Mot de passe", "Numéro de téléphone", "Nom de société", and "Votre rôle dans l'entrep". A dark "S'inscrire" button is at the bottom, with a link "Déjà un compte ? Se connecter" below it.

Figure 21: Inscription



The screenshot shows a login form titled "Connexion" on a dark background. The form is a light beige rounded rectangle containing "Email" and "Mot de passe" input fields, followed by a dark "Se connecter" button. A link "Pas de compte ? Créer un compte" is at the bottom.

Figure 22: Connexion

— **Interface de chat** : espace principal de l'utilisateur standard, avec zone de conversation où les réponses s'affichent en streaming, accompagnées des sources et d'un bouton d'export. Une barre latérale donne accès à l'historique des conversations et à l'import de documents, avec le nombre de segments indexés et la date d'import.

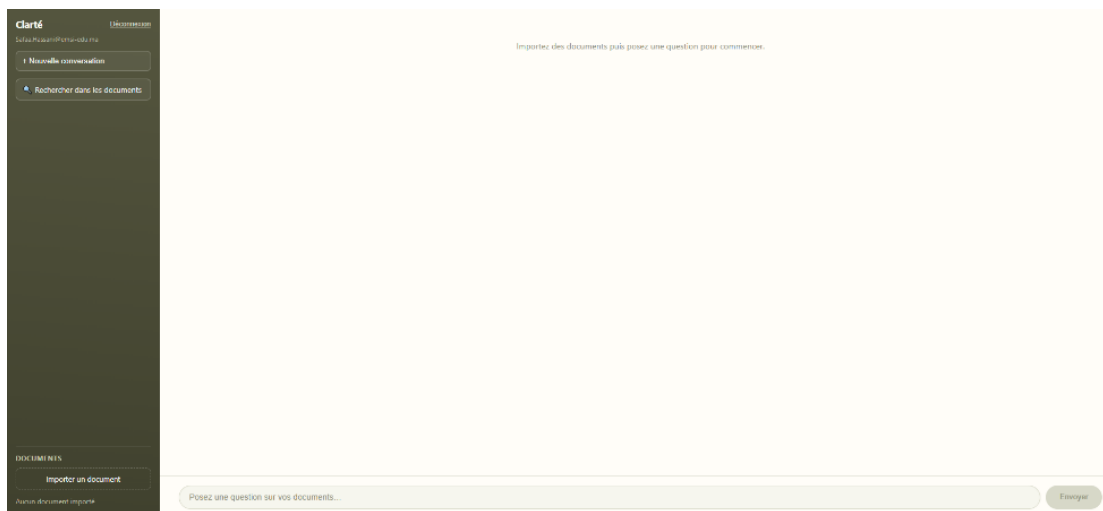


Figure 23: Interface de chat

— Page de recherche sémantique : contrairement au chat, elle n’appelle pas le LLM et renvoie directement les passages les plus pertinents pour une requête, classés et accompagnés du document source — utile pour retrouver un extrait rapidement, sans attendre de réponse générée.

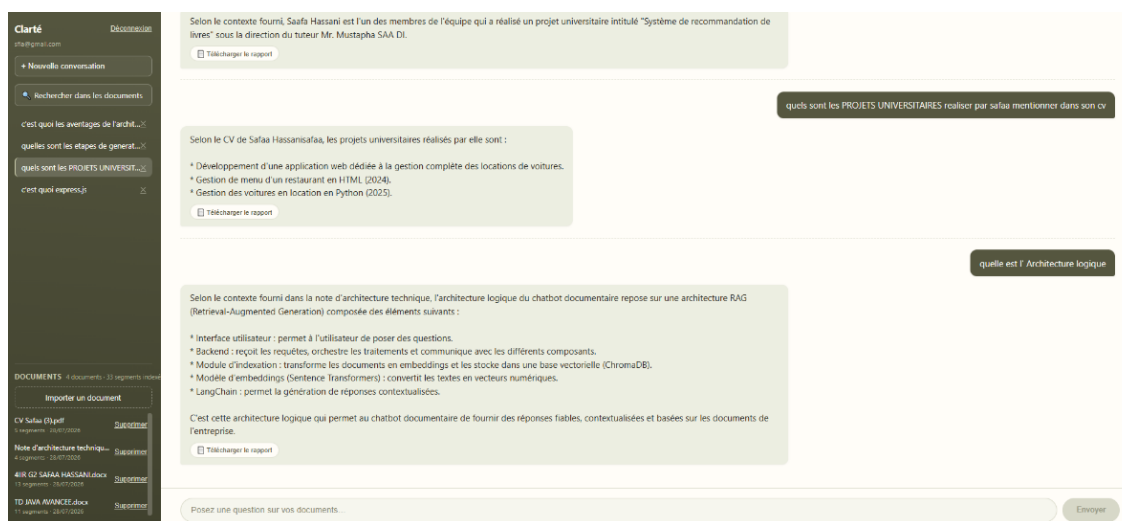


Figure 24: Page de recherche sémantique

Rapport généré

— À l’issue de chaque échange avec l’assistant, l’utilisateur a la possibilité de télécharger un rapport structuré résumant l’interaction. Ce rapport, généré automatiquement au format texte, reprend la question posée, la réponse fournie par le système ainsi que les sources documentaires ayant permis de la construire. Il offre ainsi une traçabilité complète de chaque réponse et permet à l’utilisateur de conserver ou de partager facilement une information vérifiée, sans avoir à recopier manuellement le contenu de la conversation.

```

=====
RAPPORT DE RÉPONSE – CLARTÉ
=====

Généré le : 02/09/2026 18:18:05
Utilisateur : sfia@gmail.com

1. QUESTION

Supprime un département du set (méthode statique).

2. RÉPONSE

Voici la méthode statique qui permet de supprimer un département du set :

```java
public static void supprimerDepartement(Set<Departement> lesDeps, Departement d) {
 if (lesDeps.remove(d)) {
 // On traite la relation en mettant la référence du gérant à null
 d.setGerant(null);
 } else {
 System.out.println("Erreur : Le département " + d.getNom() + " n'existe pas dans le set.");
 }
}
```

Notez que nous supprimons le département du set en utilisant la méthode `remove()` de l'interface `Set`.
Si le département est trouvé et supprimé, nous mettons la référence du gérant à null pour traiter la relation
Many-to-One entre un département et son gérant.

3. SOURCE

TD JAVA AVANCEE.docx, rapport projet react.pdf
=====

```

Figure 25: Rapport génère

— **Tableau de bord administrateur : espace séparé de l’interface utilisateur, sans accès au chat. Il regroupe des indicateurs d’usage (utilisateurs, documents, conversations), la gestion des comptes (création, modification, désactivation, suppression, avec garde-fous) et la gestion de l’ensemble des documents importés par les utilisateurs**

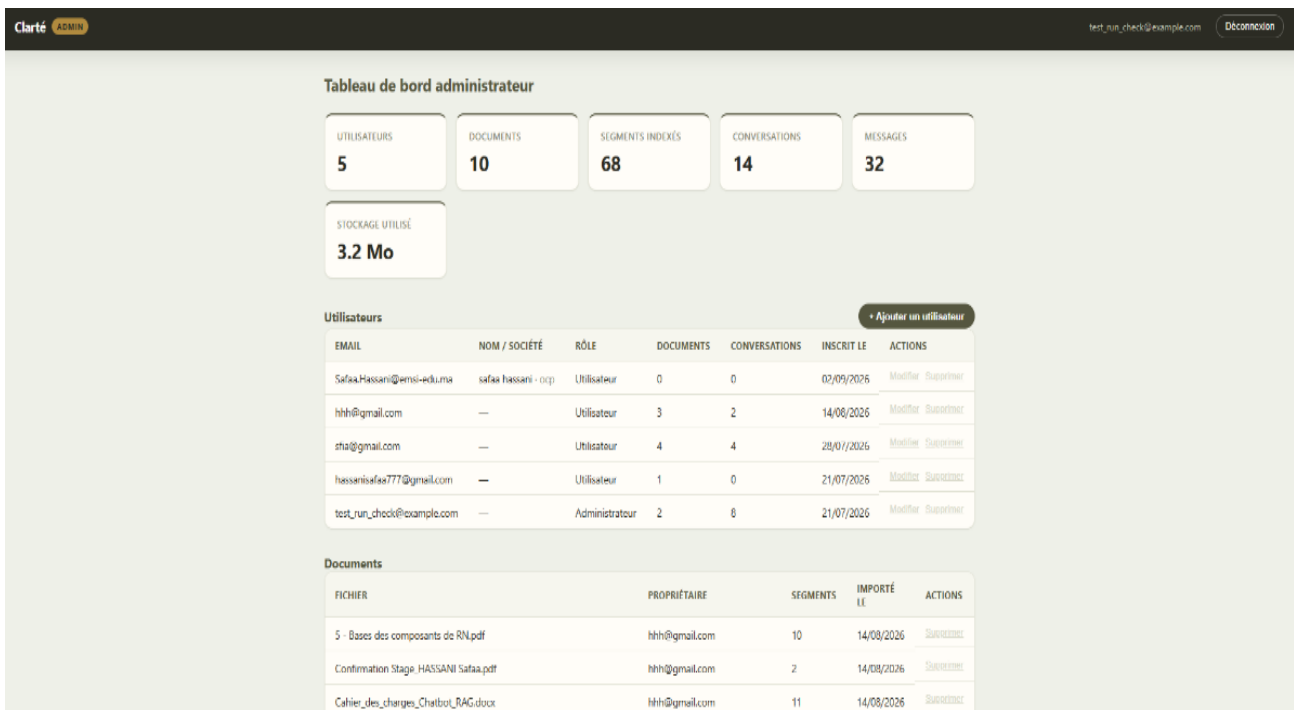


Figure 26: Tableau de bord administrateur

Test de réalisation :

The screenshot shows the Clarté web application interface. On the left is a dark sidebar with the user's name 'Clarté', email 'test@gmail.com', and a 'Déconnexion' link. Below this are buttons for '+ Nouvelle conversation', 'Rechercher dans les documents', and a search bar containing 'Combien de jours dure le congé p...'. The main content area has a light yellow background and displays search results for the query 'Combien de jours dure le congé paternité ?'. The first result shows '25 jours calendaires.' with a 'Télécharger le rapport' button. The second result, titled 'Combien de jours de télétravail par semaine sont autorisés ?', provides information about the company's teleworking policy and also has a 'Télécharger le rapport' button. The third result, titled 'Entre quelles heures un télétravailleur doit-il être joignable ?', shows 'De 9h à 18h.' with a 'Télécharger le rapport' button. The fourth result, titled 'À partir de quel jour d'absence un justificatif médical est-il requis', shows 'À partir du 3e jour d'absence consécutif.' with a 'Télécharger le rapport' button. At the bottom, there is a search bar with the text 'Posez une question sur vos documents...' and an 'Envoyer' button. A 'DOCUMENTS' section on the left sidebar lists two documents: 'politique_conges.txt' and 'politique_teletravail.txt', each with a 'Supprimer' button.

Figure 27: Test de réalisation

| Aspect évalué | Indicateur | Résultat | Interprétation simple |
|--------------------------|-------------------------|------------------|---|
| Qualité du retrieval | Hit Rate@2 | 100 % | Le bon document a été retrouvé pour les 12 questions. |
| Qualité du retrieval | MRR | 1,000 | Le document pertinent est toujours classé en première position. |
| Fidélité des réponses | Réponses fidèles | 100 %
(12/12) | Les réponses contiennent le fait demandé ou refusent correctement de répondre si l'information est absente. |
| Fidélité des réponses | Réponses non fondées | 0 | Aucune information inventée n'a été détectée dans ce jeu de test. |
| Exactitude des citations | Rappel des citations | 100 % | La bonne source est toujours affichée avec la réponse. |
| Exactitude des citations | Précision des citations | 58 % | Certaines réponses affichent aussi un document supplémentaire, non nécessaire. |
| Exactitude des citations | Citations exactes | 33 %
(4/12) | Seules 4 réponses sur 12 affichent uniquement la ou les sources attendues. |
| Performance | Latence moyenne | 9,1 s | Le temps moyen nécessaire pour générer une réponse est d'environ neuf secondes. |

Les résultats montrent que le système RAG récupère efficacement les documents pertinents et produit des réponses fidèles au corpus fourni. Le principal point à améliorer concerne les citations : la source correcte est toujours présente, mais le système affiche parfois des documents supplémentaires. Une amélioration possible serait de filtrer les sources affichées afin de ne conserver que les passages réellement utilisés pour générer.

4.4 Conclusion

Ce chapitre a montré le passage de la conception à un produit concret : l'environnement matériel et logiciel a été précisé, les principales interfaces graphiques ont été présentées, et l'application qui en résulte couvre l'ensemble du pipeline RAG ainsi que les fonctionnalités transverses attendues d'une application d'entreprise — authentification, gestion documentaire, historique des échanges et administration dédiée.

Affirmer qu'une application « fonctionne » ne suffit cependant pas à en établir la qualité : encore faut-il le mesurer. C'est l'objet du harnais d'évaluation quantitative développé spécifiquement pour Clarté , qui interroge le pipeline réel — embeddings, ChromaDB, Ollama — sur un jeu de questions dont la réponse et la source attendues sont connues à l'avance, plutôt que de se fier à une appréciation subjective des réponses obtenues.

Conclusion générale

Ce projet de fin d'année a porté sur la conception et le développement de Clarté, un chatbot intelligent d'entreprise fondé sur les techniques de Retrieval-Augmented Generation. Prototype fonctionnel— permettre à un collaborateur d'obtenir, en langage naturel, une réponse fiable et systématiquement sourcée à partir des documents internes de l'entreprise — a été atteint : l'application couvre l'ensemble de la chaîne fonctionnelle, de l'import et l'indexation des documents jusqu'à la génération de réponses en temps réel, en passant par la recherche sémantique autonome, l'historique des échanges et un espace d'administration dédié au suivi de l'usage.

Sur le plan technique, ce projet a permis de mettre en œuvre concrètement une architecture RAG complète (extraction, découpage, embeddings , indexation vectorielle, retrieval , génération), tout en intégrant des exigences propres à un contexte d'entreprise : isolation stricte des données par utilisateur, exécution locale du modèle de langage pour préserver la confidentialité, séparation des privilèges administrateur/utilisateur, et amélioration continue de l'expérience utilisateur (réponses en streaming, téléchargement de rapports, limitation du temps de génération). Plusieurs axes d'amélioration ont été identifiés et pourraient faire l'objet de travaux futurs : l'élargissement du jeu d'évaluation quantitative à un corpus plus large et plus varié, la mise en place de tests automatisés, la conteneurisation de l'application pour faciliter son déploiement ou encore l'ajout d'une fonctionnalité de récupération de mot de passe en libre-service.

Bibliographie

FastAPI. (s.d.). *FastAPI documentation*. Consulté le 10 septembre 2026, à l'adresse <https://fastapi.tiangolo.com/>

Uvicorn. (s.d.). *Uvicorn documentation*. Consulté à l'adresse <https://www.uvicorn.org/>

LangChain. (s.d.). *LangChain documentation*. Consulté à l'adresse <https://python.langchain.com/>

Chroma. (s.d.). *ChromaDB documentation*. Consulté à l'adresse <https://docs.trychroma.com/>

Ollama. (s.d.). *Ollama documentation*. Consulté à l'adresse <https://ollama.com/>

Meta AI. (s.d.). *Llama 3 model*. Consulté à l'adresse <https://llama.meta.com/llama3/>

SBERT. (s.d.). *Sentence-Transformers documentation — all-MiniLM-L6-v2*. Consulté à l'adresse <https://www.sbert.net/>

Hugging Face. (s.d.). *Model hub and langchain-huggingface library*. Consulté à l'adresse <https://huggingface.co/>

SQLAlchemy. (s.d.). *SQLAlchemy documentation*. Consulté à l'adresse <https://docs.sqlalchemy.org/>

Pydantic. (s.d.). *Pydantic documentation*. Consulté à l'adresse <https://docs.pydantic.dev/>

Davis, M. (s.d.). *python-jose: JWT library for Python* [Logiciel]. GitHub. <https://github.com/mpdavis/python-jose>

Passlib. (s.d.). *Passlib documentation — password hashing library*. Consulté à l'adresse <https://passlib.readthedocs.io/>

React. (s.d.). *React documentation*. Consulté à l'adresse <https://react.dev/>

Vite. (s.d.). *Vite documentation*. Consulté à l'adresse <https://vitejs.dev/>

React Router. (s.d.). *React Router documentation*. Consulté à l'adresse <https://reactrouter.com/>

Axios. (s.d.). *Axios documentation*. Consulté à l'adresse <https://axios-http.com/>

MDN Web Docs. (s.d.). *Fetch API et ReadableStream*. Mozilla. Consulté à l'adresse <https://developer.mozilla.org/>

Git. (s.d.). *Git documentation*. Consulté à l'adresse <https://git-scm.com/>

Chapitre A

Annexes

[Facultatif. Peuvent y figurer : extraits de code jugés significatifs, diagrammes UML complémentaires, interfaces graphiques additionnelles.]